

StepbyStep Guide

Step-by-Step Build Guide — MacroSnap

MacroSnap is the flagship project built live in this workshop: a Streamlit chat app where a student enters their name and WhatsApp number once, then chats with an AI nutrition buddy - typing a question or attaching a photo of a meal - and gets an instant calorie/macro estimate. One button sends a full summary of the conversation straight to WhatsApp. It's built with Gemini (chat + vision) and Twilio (WhatsApp) - no OpenCV, no MediaPipe, no model training.

What you'll need before building the project

- Python 3.9 or newer installed
- Basic comfort with Python (this is the one prerequisite for students)
- A free Google AI Studio account, for a Gemini API key
- A free Twilio account, for the WhatsApp sandbox

Project structure

```
macrosnap/  
├── app.py                # the app itself  
├── prompts.py            # the AI's personality, kept separate  
├── requirements.txt      # dependencies  
├── .gitignore            # keeps secrets.toml out of GitHub  
├── .streamlit/  
│   └── secrets.toml.example # template - copy to secrets.toml and fill in
```

Step 1 — Environment & keys

1. Create a project folder, e.g. macrosnap/
2. Create a virtual environment: `python -m venv venv`
3. Activate it - macOS/Linux: `source venv/bin/activate` · Windows PowerShell: `.\venv\Scripts\Activate.ps1` (run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` first if blocked) · Windows cmd.exe: `venv\Scripts\activate.bat`
4. Install dependencies: `pip install -r requirements.txt`
5. Get a Gemini API key from Google AI Studio (aistudio.google.com → Get API key)
6. Sign up free at twilio.com/try-twilio, then from the Console copy your Account SID and Auth Token
7. Under Messaging → Try it out → Send a WhatsApp message, find your sandbox number and join code (e.g. "join happy-tiger")
8. From the WhatsApp number you'll test with, send that join message to the sandbox number - this opt-in is required and expires after ~72 hours of inactivity
9. In the Twilio Console, go to Messaging → Content Template Builder and create a Text template with one variable for the recipient's name and one for the summary (e.g. "Hi {{1}}, here's your MacroSnap summary:\n\n{{2}}") - note its Content SID (starts with HX...)

Why a Content Template? WhatsApp's business-messaging policy only allows free-form text replies within a 24-hour window opened by the customer. A message your app sends on its own initiative - like our WhatsApp summary button - counts as business-initiated, so Twilio requires an approved template rather than arbitrary text.

Step 2 — requirements.txt

```
streamlit>=1.63.0
google-generativeai>=0.3.0
twilio>=9.11.0
```

Step 3 — prompts.py: giving MacroSnap a personality

Keeping prompts in their own file separates the AI's "personality" from the app's logic, so either can be tweaked independently.

```
SYSTEM_PROMPT = """You are MacroSnap, a friendly AI nutrition buddy.
Your ONLY job is to help the user understand what they're eating -
estimating calories and macros from a photo or a text description.

If the user asks about anything unrelated to food, nutrition, meals, or
fitness, politely decline and steer the conversation back to food.

When estimating a meal from a photo or description, always include:
1. What the meal appears to be
2. Estimated calories
3. Estimated protein / carbs / fat (rough is fine - say so)

Keep replies short, friendly, and conversational - no markdown formatting."""

WELCOME_MESSAGE_TEMPLATE = (
    "Hey {name}! I'm MacroSnap 🥗 - your instant calorie & macro decoder.\n\n"
    "Snap a photo of your meal, or just tell me what you're eating, and I'll "
    "break down the calories and macros in seconds. No food diary, no "
    "guesswork.\n\n"
    "When you're done, hit \"Send details to WhatsApp\" below and I'll text "
    "your full summary straight to your phone."
)

SUMMARY_REQUEST_PROMPT = (
    "Summarize every meal we've discussed in this conversation into one "
    "WhatsApp-friendly message: list each item with its estimated calories, "
    "then give a running total of calories and macros (protein/carbs/fat) "
    "for everything combined. Keep it short, plain text with a couple of "
    "emojis, no markdown - ready to send exactly as you write it."
)
```

- **SYSTEM_PROMPT** - sets the persona and, critically, the instruction that keeps replies on-topic (food/fitness only). This is set once per conversation via Gemini's `system_instruction`, so it applies to every message without any extra if-statements in `app.py`.
- **WELCOME_MESSAGE_TEMPLATE** - the first message a student sees; `{name}` gets filled in after onboarding.
- **SUMMARY_REQUEST_PROMPT** - not shown to the user. It's sent to Gemini behind the scenes when the WhatsApp button is clicked, asking it to reread the whole conversation and produce one clean summary.

Step 4 — Connecting to Gemini

Build the Gemini client once, cached, so it survives every Streamlit rerun:

```
@st.cache_resource
def get_gemini_client():
    return genai.Client(api_key=GEMINI_API_KEY)

gemini_client = get_gemini_client()
```

Gotcha to flag live: Streamlit re-executes the entire script top to bottom on every interaction. If `genai.Client(...)` is created as a plain top-level variable, a brand-new client is built on every rerun and the previous one is garbage-collected - closing its connection. Any chat session created from that old client then fails with "Cannot send a request, as the client has been closed." Wrapping creation in `@st.cache_resource` builds the client once per session and reuses it, so every chat and every message shares the same live connection.

Step 5 — Building the onboarding screen

```
if "onboarded" not in st.session_state:
    st.title("📸 MacroSnap")
    st.caption("Snap it. Track it. Text yourself the results.")
    with st.form("onboarding_form"):
        name = st.text_input("Your name")
        whatsapp_number = st.text_input(
            "WhatsApp number (with country code)",
            placeholder="+91XXXXXXXXXX",
        )
        submitted = st.form_submit_button("Let's go 🚀")
    if submitted:
        if not name.strip() or not whatsapp_number.strip():
            st.warning("Please fill in both your name and WhatsApp number.")
        else:
            st.session_state.name = name.strip()
            st.session_state.whatsapp_number = whatsapp_number.strip()
            st.session_state.chat = gemini_client.chats.create(
                model=MODEL_NAME,
                config=types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT),
            )
            st.session_state.messages = []
            st.session_state.onboarded = True
            st.rerun()
    st.stop()
```

- if "onboarded" not in st.session_state is the standard Streamlit pattern for "show this only once per session".
- Submitting the form is where the actual Gemini chat session is born - `gemini_client.chats.create(...)` with `system_instruction=SYSTEM_PROMPT` gives the conversation its memory and its personality, and it's stored in session state so every later message reuses the same conversation.
- `st.rerun()` forces Streamlit to immediately restart the script so it picks up `onboarded = True` and skips straight past the form next time.

Step 6 — Building the chat interface

```
def render_message(message):
    with st.chat_message(message["role"]):
        if message["kind"] == "text":
            st.write(message["content"])
        elif message["kind"] == "image":
```

```

        st.image(message["content"])

def add_message(role, kind, content):
    st.session_state.messages.append({"role": role, "kind": kind, "content":
content})
    render_message(st.session_state.messages[-1])

# ...

if not st.session_state.messages:
    add_message("assistant", "text",
WELCOME_MESSAGE_TEMPLATE.format(name=st.session_state.name))
else:
    for message in st.session_state.messages:
        render_message(message)

```

- Every message is stored as a small dict with a role (user/assistant), a kind (text/image) and the content - one `render_message` function handles both a typed question and an uploaded photo the same way.
- `add_message` does two things at once: saves the message to history and draws it on screen immediately.
- The welcome message only fires the very first time the chat is empty; after that, history is replayed from session state on every rerun.

Step 7 — Handling input: text and photos

```

def ask_gemini(parts):
    try:
        return st.session_state.chat.send_message(parts).text
    except Exception as error:
        return f"Sorry, something went wrong: {error}"

user_input = st.chat_input(
    "Ask a question, or attach a photo of your meal",
    accept_file=True,
    file_type=["jpg", "jpeg", "png"],
)

if user_input:
    photo = user_input.files[0] if user_input.files else None
    text = user_input.text
    parts = []

    if photo is not None:
        photo_bytes = photo.getvalue()
        add_message("user", "image", photo_bytes)
        parts.append(types.Part.from_bytes(data=photo_bytes, mime_type=photo.type))
    if text:
        add_message("user", "text", text)
        parts.append(text)
    elif photo is not None:
        parts.append("What is this meal? Give me the calories and macros.")

    with st.spinner("Crunching the numbers..."):
        answer = ask_gemini(parts)
    add_message("assistant", "text", answer)

```

- `st.chat_input(accept_file=True, ...)` is one input box that takes either typed text or an attached photo - the Google-Lens-style feel comes from here.

- `types.Part.from_bytes(...)` is the actual vision step: it hands the raw image bytes and mime type to Gemini.
- If a photo is attached with no caption, the code quietly supplies its own instruction, so a bare photo still works.
- Because `ask_gemini` always calls the same `st.session_state.chat`, every message stacks onto one conversation - a follow-up question like "how much protein was in that?" just works.

Step 8 — Sending the summary to WhatsApp

```
def clean_whatsapp_text(text):
    if not text:
        return "No nutrition summary available."
    text = " ".join(text.split())
    return text[:1500] + "..." if len(text) > 1500 else text

def send_whatsapp(to_number, user_name, summary):
    # Content template expects {{1}} = name, {{2}} = summary.
    try:
        content_variables = json.dumps(
            {"1": user_name, "2": clean_whatsapp_text(summary)}, ensure_ascii=False
        )
        message = twilio_client.messages.create(
            from_=TWILIO_WHATSAPP_FROM,
            to=f"whatsapp:{to_number}",
            content_sid=TWILIO_CONTENT_SID,
            content_variables=content_variables,
        )
        return True, message.sid
    except Exception as error:
        return False, str(error)

header_col, button_col = st.columns([5, 2], vertical_alignment="center")

with header_col:
    st.title("🥗 MacroSnap")

with button_col:
    send_disabled = len(st.session_state.messages) <= 2
    if st.button("📱 Send to WhatsApp", disabled=send_disabled,
use_container_width=True):
        with st.spinner("Summarizing your day..."):
            summary = ask_gemini([SUMMARY_REQUEST_PROMPT])
            success, info = send_whatsapp(st.session_state.whatsapp_number,
st.session_state.name, summary)
            if success:
                st.success("Sent! Check your WhatsApp 📱")
            else:
                st.error(f"Couldn't send that: {info}")
```

- The button sits beside the title, in a `st.columns([5, 2], vertical_alignment="center")` layout.
- `send_disabled = len(st.session_state.messages) <= 2` keeps the button greyed out until there's an actual logged exchange (not just the welcome message) - a disabled button can't be clicked, so there's no need for a separate "log a meal first" warning.
- The click handler reuses `ask_gemini`, but this time sends the hidden `SUMMARY_REQUEST_PROMPT` - Gemini rereads everything said in the conversation and writes one clean recap.
- `send_whatsapp` sends via Twilio's Content API (`content_sid` + `content_variables`) instead of a plain body string, because WhatsApp requires an approved template for this kind of business-initiated message (see Step 1's note).

Full app.py (reference)

The complete file, top to bottom, exactly as it should look after all eight steps above:

```
import json

import streamlit as st
from google import genai
from google.genai import types
from twilio.rest import Client as TwilioClient

from prompts import SUMMARY_REQUEST_PROMPT, SYSTEM_PROMPT, WELCOME_MESSAGE_TEMPLATE

MODEL_NAME = "gemini-3.5-flash"
st.set_page_config(page_title="MacroSnap", page_icon="🥗")

GEMINI_API_KEY = st.secrets["GEMINI_API_KEY"]
TWILIO_ACCOUNT_SID = st.secrets["TWILIO_ACCOUNT_SID"]
TWILIO_AUTH_TOKEN = st.secrets["TWILIO_AUTH_TOKEN"]
TWILIO_WHATSAPP_FROM = st.secrets["TWILIO_WHATSAPP_FROM"]
TWILIO_CONTENT_SID = st.secrets["TWILIO_CONTENT_SID"]

@st.cache_resource
def get_gemini_client():
    return genai.Client(api_key=GEMINI_API_KEY)

@st.cache_resource
def get_twilio_client():
    return TwilioClient(TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN)

gemini_client = get_gemini_client()
twilio_client = get_twilio_client()

def render_message(message):
    with st.chat_message(message["role"]):
        if message["kind"] == "text":
            st.write(message["content"])
        elif message["kind"] == "image":
            st.image(message["content"])

def add_message(role, kind, content):
    st.session_state.messages.append({"role": role, "kind": kind, "content": content})
    render_message(st.session_state.messages[-1])

def ask_gemini(parts):
    try:
        return st.session_state.chat.send_message(parts).text
    except Exception as error:
        return f"Sorry, something went wrong: {error}"

def clean_whatsapp_text(text):
    if not text:
        return "No nutrition summary available."
    text = " ".join(text.split()) # collapse whitespace/newlines
    return text[:1500] + "..." if len(text) > 1500 else text
```

```

def send_whatsapp(to_number, user_name, summary):
    # Content template expects {{1}} = name, {{2}} = summary.
    try:
        content_variables = json.dumps(
            {"1": user_name, "2": clean_whatsapp_text(summary)}, ensure_ascii=False
        )
        message = twilio_client.messages.create(
            from_=TWILIO_WHATSAPP_FROM,
            to=f"whatsapp:{to_number}",
            content_sid=TWILIO_CONTENT_SID,
            content_variables=content_variables,
        )
        return True, message.sid
    except Exception as error:
        return False, str(error)

# Step 1: onboarding
if "onboarded" not in st.session_state:
    st.title("🥰 MacroSnap")
    st.caption("Snap it. Track it. Text yourself the results.")
    with st.form("onboarding_form"):
        name = st.text_input("Your name")
        whatsapp_number = st.text_input(
            "WhatsApp number (with country code)",
            placeholder="+91XXXXXXXXXX",
            help="This is the number MacroSnap will text your summary to.",
        )
        submitted = st.form_submit_button("Let's go 🚀")
    if submitted:
        if not name.strip() or not whatsapp_number.strip():
            st.warning("Please fill in both your name and WhatsApp number.")
        else:
            st.session_state.name = name.strip()
            st.session_state.whatsapp_number = whatsapp_number.strip()
            st.session_state.chat = gemini_client.chats.create(
                model=MODEL_NAME,
            )
            config=types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT),
            )
            st.session_state.messages = []
            st.session_state.onboarded = True
            st.rerun()
    st.stop()

# Step 2: chat interface
header_col, button_col = st.columns([5, 2], vertical_alignment="center")

with header_col:
    st.title("🥰 MacroSnap")

with button_col:
    send_disabled = len(st.session_state.messages) <= 2
    if st.button("📱 Send to WhatsApp", disabled=send_disabled,
        use_container_width=True):
        with st.spinner("Summarizing your day..."):
            summary = ask_gemini([SUMMARY_REQUEST_PROMPT])
            success, info = send_whatsapp(st.session_state.whatsapp_number,
            st.session_state.name, summary)
            if success:
                st.success("Sent! Check your WhatsApp 📱")

```

```

        else:
            st.error(f"Couldn't send that: {info}")

st.caption(f"Logged in as {st.session_state.name} - updates go to {st.session_state.whatsapp_number}")

if not st.session_state.messages:
    add_message("assistant", "text",
WELCOME_MESSAGE_TEMPLATE.format(name=st.session_state.name))
else:
    for message in st.session_state.messages:
        render_message(message)

user_input = st.chat_input(
    "Ask a question, or attach a photo of your meal",
    accept_file=True,
    file_type=["jpg", "jpeg", "png"],
)

if user_input:
    photo = user_input.files[0] if user_input.files else None
    text = user_input.text
    parts = []

    if photo is not None:
        photo_bytes = photo.getvalue()
        add_message("user", "image", photo_bytes)
        parts.append(types.Part.from_bytes(data=photo_bytes, mime_type=photo.type))
    if text:
        add_message("user", "text", text)
        parts.append(text)
    elif photo is not None:
        parts.append("What is this meal? Give me the calories and macros.")

    with st.spinner("Crunching the numbers..."):
        answer = ask_gemini(parts)
    add_message("assistant", "text", answer)

```

[.streamlit/secrets.toml.example](#)

Copy this to `.streamlit/secrets.toml` (drop the `.example`) and fill in real values. Never commit the real `secrets.toml` - only this template file.

```

GEMINI_API_KEY = "your-gemini-api-key-here"

# From your Twilio Console (console.twilio.com):
TWILIO_ACCOUNT_SID = "your-twilio-account-sid-here"
TWILIO_AUTH_TOKEN = "your-twilio-auth-token-here"

# The Twilio Sandbox's shared WhatsApp number - leave as-is unless you have
# your own approved WhatsApp sender.
TWILIO_WHATSAPP_FROM = "whatsapp:+14155238886"

# The Content SID of your WhatsApp Content Template (Twilio Console >
# Messaging > Content Template Builder). Required because WhatsApp does not
# allow plain free-text messages for business-initiated sends.
TWILIO_CONTENT_SID = "your-content-template-sid-here"

```

Running it locally

```
streamlit run app.py
```

This opens the app at <http://localhost:8501>. Onboard with the WhatsApp number that joined your sandbox, try a text question, try a photo, then click the WhatsApp button once you've logged something.

Deploying — Streamlit Community Cloud

1. Push the project to a GitHub repo. Do not commit `.streamlit/secrets.toml` - only `secrets.toml.example` should be tracked.
2. Go to share.streamlit.io and sign in with GitHub.
3. Click "New app", pick the repo, branch, and `app.py` as the entry point.
4. In the app's Settings → Secrets panel, paste the same content your local `secrets.toml` has.
5. Deploy - you get a public URL, no separate server to manage.

Projects

Project Ideas for you to Build

Everyone builds MacroSnap together in the live session. For their own project, students pick one of the ideas below - or design their own, using the same building blocks: Gemini vision + chat, and WhatsApp (via Twilio) as the action tool.

1. Snap & Study · Easy

- Student photographs a problem, diagram, or page of notes they don't understand.
- Gemini explains it in plain language and breaks down the key concept.
- A "send to WhatsApp" button texts the explanation to the student, so it's saved outside the app.

What you will learn:

- Prompting Gemini for clear, structured explanations rather than just descriptions
- Reusing the onboard → chat → WhatsApp-send pattern from MacroSnap for a different use case

2. Receipt & Expense Tracker / Bill Splitter · Easy

- Student photographs a receipt or bill.
- Gemini reads the itemized amounts and total, and (optionally) splits it evenly or by item across a number of people.
- The breakdown is sent to WhatsApp - handy as a quick "emergency bill splitter."

What you will learn:

- Getting structured data (amounts, totals) out of a photo, not just a description
- Turning a real everyday chore - splitting a bill - into a 10-second automation

3. Deadline Tracker · Medium

- Student photographs a syllabus, timetable, or assignment sheet.
- Gemini extracts the dates and deadlines it can find.
- A digest of upcoming deadlines is sent to WhatsApp as a reminder.

What you will learn:

- Extracting dates/structured lists reliably from messy real-world photos
- Designing a prompt that fails gracefully when the photo doesn't contain what's expected

Your Own Idea

Prefer to build something else? Any idea is fine as long as it follows the same shape as everything above:

1. Pick a photo (or short text) a person would realistically want AI to look at - a menu, a plant, a whiteboard, a product label, anything.
2. Write a system prompt in prompts.py that keeps Gemini scoped to that one job, the same way MacroSnap's system prompt keeps it on food and fitness.
3. Reuse the onboarding → chat → WhatsApp-send pattern - swap out what the chat is about, not the structure.
4. Pick one clear "action" for the WhatsApp button to trigger - a summary, a reminder, a list, a recommendation.

Submission Guidelines

Submission Guidelines

What to submit

- A public (or shared) GitHub repository link containing your project code.
- A live deployed app link (Streamlit Community Cloud).

What your repository should include

- app.py and prompts.py (or your renamed equivalents)
- requirements.txt
- A .streamlit/secrets.toml.example template - never your real secrets.toml
- A .gitignore that excludes .streamlit/secrets.toml, venv/, and __pycache__/
- A short README: what your app does, and how to run it locally

Deadline

4th October, 11:59PM

Where to submit

<https://forms.ccbp.in/ai-vision-chatbot-last-project-submission>

Evaluation criteria

- Functionality (40%) - does the app run, and does the core flow (input → AI response → WhatsApp send) work end to end?
- Prompt design (20%) - is the system prompt scoped sensibly to the app's stated purpose?
- Code quality (20%) - is the code organized, readable, and free of committed secrets?
- Creativity / polish (20%) - thoughtful extensions, a clear README, a nice user experience.

Before submitting, double-check that .streamlit/secrets.toml (with real API keys) is not committed to the repository - only secrets.toml.example should be tracked.